

Basics B1-B6

**Interview Q&A Study Guide (with answers, new_x=XPos.LMARGIN,
new_y=YPos.NEXT)**

Coding-Agent Capability Portfolio - Task_Eval
repo-structure-mapper . route-api-mapper . test-discovery
fastapi-transaction-service . node-transaction-service . rust-logcount-cli

B1 - repo-structure-mapper

Repository inventory, architecture detection & dependency graph (analysis agent)

Concept & scope

Q. What does this agent produce and who consumes it?

A. A structured read of an unfamiliar repo: stack, architecture pattern + layers, module/dependency graph, design patterns, artifact inventory, an onboarding guide and a code-review cheat sheet. Consumers: an engineer onboarding on day 1, a reviewer needing context, or an AI/RAG index. It is a reading, not runnable code.

Q. Inventory vs architecture map vs dependency graph - why all three?

A. Inventory = what exists (files/counts). Architecture map = how layers relate and which patterns are used. Dependency graph = directional edges between modules derived from imports/build files. Together they answer what is here, how it is organized, and what depends on what.

Q. What are the depth modes and when do you pick each?

A. quick (stack + entrypoints), standard (default: + architecture + light static analysis), full (+ business-flow traces), onboarding (+ day-1 guide), code-review (+ review cheat sheet). Pick by how much coverage the consumer needs.

Methodology & correctness

Q. How did you get the counts - enumerated or estimated?

A. Originally naming-sweep grep estimates - the core weakness: they were wrong by up to ~40% (models) and >2x (providers). Now every count ships the exact command that produced it, executed at a pinned commit (1a35bda75).

Q. VERIFIED vs INFERRED vs NOT FOUND - give an example of each.

A. VERIFIED = read directly/executed (Flutter 3.35.5 from .fvmrc:2). INFERRED = pattern/name match not fully confirmed (a folder's purpose from its name). NOT FOUND IN REPOSITORY = expected-but-absent (no BLoC, no server endpoints in a client module).

Q. Are counts file-level or class-level?

A. File-level - a lower bound on classes, since one file can hold several. Stated explicitly so a reader does not conflate files with class instances.

Q. How do you detect the architecture pattern and assign confidence?

A. Look for structural signatures: per-feature data/domain/presentation folders (Clean), StateNotifier/ViewModel + ref.watch (MVVM/Riverpod), *_bloc.dart/BlocProvider (BLoC). Confidence is VERIFIED when the signature is read in source with cited paths.

Deep-dive: the layer violation

Q. How did you prove the violation is systemic, not anecdotal?

A. Re-derived by command: 296 presentation files import a data-layer symbol (/data/, _repository_impl, _remote_datasource). That turns 'I saw one example' into a measured, structural finding.

Q. Is importing a data model as severe as importing a _repository_impl?

A. No - importing a concrete *_repository_impl couples to an implementation and is worse than importing a DTO/model. The 296 is a conservative superset; a rigorous version splits the two for action.

Q. How would you enforce the boundary going forward?

A. A custom analyzer / import-lint rule banning presentation -> data imports, wired into analysis_options.yaml and failed in CI.

Freshness, scaling & trade-offs

Q. This is a static snapshot - how does it avoid going stale?

A. Anchor to a commit SHA and ship the regenerating commands; re-run after a pull and bump the SHA. That

converts a snapshot into a reproducible artifact.

Q. How does the approach scale to a 1M-line monorepo?

A. grep/find scale fine for counts; for relationships use codegraph/LSP symbol data (call graph, refs) which is more accurate but heavier. In huge repos sample + cite representatives rather than enumerate, and say so.

Q. What if the README contradicts the code?

A. Code is the source of truth; flag the drift (README said Dio, code uses package:http) as documentation drift, not a bug.

B2 - route-api-mapper

Outbound API surface of a client app (Retrofit + deep links)

Concept

Q. A client app has no server endpoints - what is the API surface?

A. The outbound surface: Retrofit service interfaces (the HTTP calls the app makes) plus navigation/deep-link routes (intent-filters dispatched by a DeeplinkActivity).

The core gotcha: @Url

Q. Every method uses a dynamic @Url - what does that mean for mapping?

A. The annotation gives only the HTTP verb; the path is a runtime @Url string. So the annotation alone cannot tell you the endpoint path.

Q. If the path is not in the annotation, where is it?

A. full URL = env-selected base host (from the keys-provider UrlsProvider registry) + a path constant (MFCommonEndPoint, EquityDefaultEndpoints, GtmEndpoints ...). Assembled at the call site and passed to @Url.

Q. Your original map said paths were 'mostly not captured' - was that correct?

A. No - it was an under-analysis. The paths are in source as const vals, just decoupled from the annotation. Re-analysis recovered 694 path constants + 400 host bases at commit e7fc70a6.

Q. A constant is /mandate/api/v4/otm/%s/... - what is the %s?

A. A printf-style placeholder for a runtime parameter (e.g. user/mandate id). Document the template; the concrete value is request-time.

Completeness

Q. Which services are NOT in the repo and how do you represent that?

A. 8+ Retrofit services are DI-injected from binary modules (equity/kyc/apprating) - interfaces NOT FOUND IN REPOSITORY. Represented honestly as out-of-source, not guessed.

Q. You recovered paths but not the method->constant binding - why is that hard?

A. The extractor lists path constants but not which Retrofit method consumes which constant; that binding lives at the call site and needs AST/call-site tracing. Stated as an open limitation.

Q. How is auth handled per endpoint?

A. Auth is injected globally by an OkHttp interceptor in the binary core SDK, so per-endpoint scope is not visible in source; documented as 'interceptor'.

Reproducibility & tooling

Q. Why emit YAML + an extractor script instead of a prose table?

A. A diffable, machine-checkable artifact: extract_routes.sh regenerates it, and diff <(extract) committed.yaml flags API drift in CI. Prose cannot be diffed or re-verified.

Q. How robust is a grep-based extractor - what does it miss?

A. It misses concatenated/runtime-built paths and templated strings. An AST parser would be more precise but heavier - trade-off accepted and documented.

Q. How would you map request/response schemas, not just paths?

A. Follow each call's request/response DTO model classes - a deeper pass beyond path recovery.

B3 - test-discovery

Frameworks, the canonical CI command, coverage & execution

Concept

Q. What is the goal, and why is the canonical command the key output?

A. Tell an engineer exactly how to run the tests the way CI does. The canonical command is the single most actionable deliverable.

Q. Why split Agent Findings from Verified Findings?

A. Agent Findings = read from config; Verified Findings = commands actually executed. The split stops inference being passed off as fact.

The execution gap (the big one)

Q. Why is 'discovered but not run' a weak deliverable?

A. Without running you cannot prove the command works, nor report pass/fail, coverage, or flakiness. The first version explicitly reported no green/red status.

Q. You ran :wsclient and got 396 tests / 21 failing - how?

A. Ran ./gradlew :wsclient:testDevelopmentDebugUnitTest and parsed the JUnit XML in build/test-results: 33 suites, 396 tests, 21 failures, 0 errors.

Q. The dry-run said testDebugUnitTest is ambiguous - what did that teach you?

A. There is no plain debug variant; the real flavored tasks are testDevelopmentDebugUnitTest (+ preprod/production/staging), confirming the canonical command and that the app has 4 flavors.

Q. Why not run all 31 modules?

A. A full cold multi-module Android+Flutter build is heavy; one representative module establishes a real green/red. The scope limit is stated, not hidden, and faking a full green would be dishonest.

The :advisory finding

Q. What actually happened to :advisory?

A. Originally 'missing from settings.gradle'; now the module was removed entirely - but .gitlab-ci.yml still invokes :advisory in 3 places (lint :96, report path :132, sonarqube :173), so those jobs now fail. 'Removed but still called' is worse than 'orphaned'.

Coverage, health & freshness

Q. Is the coverage gate meaningful?

A. The JaCoCo gate is 20% globally - low, and a global rule lets a few well-tested modules mask untested ones. Per-module minimums would be stronger.

Q. The failures are in *PropertyTest - what does that suggest?

A. Property-based tests, which can be seed/environment sensitive. Reported as fact; cause not assumed without diagnosis.

Q. How did this illustrate staleness?

A. The original target repo (paytmoney) was consolidated into android-monorepo, so the pinned analysis went obsolete. Re-anchored to a new SHA with a reproduce recipe.

B4 - fastapi-transaction-service

Layered Python transaction service (FastAPI)

Architecture

Q. Walk me through the layers.

A. routes (HTTP only) -> services (business logic, balance) -> storage (persistence); plus models (domain) and schemas (API contract). Each layer depends only on the one below; routes are thin dispatchers.

Q. Why keep domain models separate from Pydantic schemas?

A. Schemas validate/serialize at the API boundary; domain models are framework-agnostic internals. Decoupling keeps storage/business code free of HTTP shapes.

Q. How does DI wire the service, and why a singleton store?

A. `get_service()` injects a `TransactionService` bound to a module-level singleton `InMemoryStorage` via FastAPI `Depends`, so all requests share state.

Money correctness (the headline)

Q. Why is float wrong for money?

A. Binary floats cannot represent decimal cents exactly ($0.1 + 0.2 = 0.30000000000000004$), so sums drift. Money needs exact decimal/integer arithmetic.

Q. `Field(gt=0)` still accepts `inf/nan/9.999` - how did you close that?

A. A field validator: must be finite (`math.isfinite`), `<= max_amount`, and at most 2 decimal places (checked via `Decimal(str(v))` exponent).

Q. Why integer cents instead of changing the field to `Decimal`?

A. It keeps the public JSON API as plain numbers (no `Decimal` serialization surprises) while guaranteeing exact aggregation: balance is summed in int cents then converted back.

Q. Why `Decimal(str(v))` and not `Decimal(v)`?

A. `Decimal(float)` imports the binary-float error; `Decimal(str(v))` parses the decimal text exactly.

Q. How do you test inf rejection when JSON has no Infinity literal?

A. The JSON number `1e500` parses to `float('inf')` server-side, exercising the `isfinite` guard - whereas `httpx` refuses to serialize `float('inf')` client-side.

Validation, errors & observability

Q. Where does validation live and why not in the route?

A. In the schemas at the boundary, so routes stay thin and can trust their inputs.

Q. Why a custom `{error, detail}` envelope, and why strip `ctx/input`?

A. For a consistent error shape; `ctx/input` can hold non-serializable values (the raw `inf` that triggered the error) which would crash `JSONResponse`.

Q. Why structured JSON logs + a request-id?

A. One JSON object per line is machine-parseable; a per-request id threaded via a `contextvar` correlates all logs for a request and is echoed in the response header.

Testing & production

Q. What is your coverage gate?

A. `pytest-cov` with `--cov-fail-under=90`; the suite sits at ~99%.

Q. Storage is in-memory - what breaks in production?

A. Data is lost on restart and not shared across processes/replicas; `threading.Lock` guards one process but each `uvicorn` worker has its own store. Production needs a real DB, auth, and rate limiting.

Q. How would you add pagination to GET /transactions?

A. Add limit/offset (or cursor) params, slice in storage, and return a total count + next cursor.

B5 - node-transaction-service

Layered Node/Express transaction service (B4 parity)

Architecture & parity

Q. This mirrors B4 - what is the same and what differs?

A. Same 3-layer split and money fix, in Express. Differences: JavaScript (manual validation, no Pydantic), a single-threaded event loop, and JSDoc instead of static types.

Q. Why is the in-memory store 'not thread-safe' but fine in practice?

A. Node's single-threaded event loop serializes the synchronous `add()`, so there is no data race in one process; the pattern is still naive for multi-process deployment.

Validation & money

Q. Show the validation checks.

A. `typeof number` and not `NaN`, finite, `> 0`, `<= maxAmount`, and at most 2 decimals via `abs(amount*100 - round(amount*100)) > 1e-9`.

Q. Why an epsilon instead of exact equality?

A. `amount*100` itself can carry tiny float error ($0.1*100 = 10.000\dots2$), so compare within $1e-9$ rather than exactly.

Q. How do `toCents/fromCents` avoid drift, and where could they fail?

A. `round(amount*100)` is lossless for `<=2dp` inputs (validated upstream); inputs outside that contract could mis-round, which is why the validation gate exists first.

Errors, logging & testing

Q. Explain the middleware order.

A. `request-id/logging` -> `express.json()` -> JSON-parse error handler -> routes -> 404 -> catch-all 500. This ensures parse errors are caught, unknown routes 404, and any thrown error returns a clean 500.

Q. How do you ensure a thrown error never leaks a stack trace?

A. The catch-all error middleware logs the error server-side and returns a generic `{error:'internal_error'}` with status 500.

Q. Why split `stdout` (info) vs `stderr` (warn/error)?

A. 12-factor stream separation, so log routing/alerting can treat error streams differently.

Q. Function coverage was under the gate - how did you fix it?

A. `logger.warn/error` were never exercised; a small unit test calling them (and the money helpers) lifted function coverage above the threshold.

Production

Q. Same in-memory limit as B4 - migration path?

A. Swap storage for Postgres behind the same interface; add auth + rate limiting; containerize and wire CI.

Q. How would you add graceful shutdown?

A. Trap `SIGTERM`, stop accepting new connections, let in-flight requests drain, then exit.

B6 - rust-logcount-cli

Deterministic Rust CLI (library + binary)

Design

Q. Why split into a library crate + a thin binary?

A. The library holds parsing/counting; the binary is a thin CLI shell. Integration tests exercise the library API directly and the binary end-to-end.

Q. Walk the layers.

A. main (argv, exit codes, I/O) -> parser (text/file -> counts) -> models (LogLevel classification, LogCounts). main carries no business logic.

Q. Why does LogLevel::from_line return Option, not error?

A. Unrecognized/blank lines are not errors; returning None lets them be skipped without panicking or failing the run.

Error handling & UX (the strength)

Q. How do you avoid panicking on a missing file?

A. count_file returns io::Result via the ? operator; main matches the error kind (NotFound -> exit 1) instead of unwrapping.

Q. What exit codes do you use and why?

A. 0 ok, 1 read/IO error, 2 usage error (missing/unknown args). Distinct codes let shell scripts branch on the failure type.

Q. Classification is case-sensitive - intended?

A. Yes - it matches exact tokens INFO/WARN/ERROR; lowercase is not counted. A flag could relax this if a use case needed it.

The improvements added

Q. Why add stdin (-) support?

A. A lone '-' reads stdin so the tool composes in pipelines (tail -f app.log | logcount -) instead of needing a temp file.

Q. Why hand-roll --json instead of using serde?

A. Counts are integers with no strings to escape, so a hand-built JSON string avoids a dependency and keeps the crate zero-deps - a deliberate trade-off vs serde's robustness.

Q. How does the parser disambiguate '-' from an unknown flag?

A. '-' exactly equals stdin; anything else starting with '-' that is not a known flag is rejected as an unknown option (exit 2).

Testing, quality & scaling

Q. How do you test the actual binary vs the library?

A. Cargo injects CARGO_BIN_EXE_logcount for integration tests; spawn it with std::process::Command, pipe stdin, and assert stdout + exit codes.

Q. What does cargo clippy -- -D warnings enforce?

A. It treats every clippy lint as an error, run in CI to keep the code idiomatic and warning-free.

Q. How would you handle a multi-GB log file?

A. Replace fs::read_to_string with a buffered line reader (BufReader::lines) so memory stays O(1) regardless of file size.

Q. Would parallelizing help?

A. Counting is I/O-bound, so threads (e.g. rayon) likely will not help unless the file is already in page cache; measure before adding complexity.

Cross-cutting

Questions asked across every Basics task

Principles

Q. Why does every task carry an 'Agent-Generated vs Verified' split?

A. It protects trust: it separates what was read/asserted from what was executed, so a reviewer knows exactly which claims are proven and which are inference.

Q. How did you behave under environment blockers?

A. Blocked or absent items are marked (blocked / NOT FOUND IN REPOSITORY) rather than fabricated - e.g. B3 did not fake a green test run when a full build was not run.

Q. For each built service, what is the difference between code-quality-ready and production-ready?

A. B4/B5/B6 are now correct, observable and tested (code-quality-ready) but still in-memory with no auth/DB/scale, so they are not deployable systems (not production-ready).

Q. What is the single highest-impact gap left per task?

A. B1: enumerate the ~46 route count. B2: resolve the method->constant binding. B3: run all modules + a real coverage %. B4/B5: real datastore + auth. B6: streaming for huge files.